



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)

B. TECH

Year: 2026 – 27

Semester: VII

Name of the Student: Swayam Mandhani

PRN: 123B1B184

Department: Computer Engineering

Division: C, **Batch** – C2

Course: DevOps

Course Code: BCE27PE01

Assignment No – 2

Title:

Study of AWS Cloud Computing Services and Practical Implementation of Amazon EC2

Aim:

To study major AWS cloud computing services including Amazon EC2, Amazon S3, AWS Lambda, Amazon RDS, Elastic Load Balancing (ELB), and Amazon ECS, and to practically create, configure, connect to, monitor, and terminate a virtual machine instance using Amazon EC2.

Theory:

Amazon EC2

Amazon Elastic Compute Cloud (EC2) provides resizable virtual computing instances. An EC2 instance can be configured with an operating system image, instance type, storage, key pair, network settings, and security group rules. For this assignment, an EC2 instance was created, accessed using EC2 Instance Connect, configured with NGINX, monitored using EC2 and CloudWatch, and finally terminated as part of the resource-cleanup process.

Amazon S3

Amazon Simple Storage Service (S3) is an object storage service. Data is stored as objects inside buckets. In this demonstration, a bucket was created, a small text object was uploaded and verified, and the resources were subsequently cleaned up.

AWS Lambda

AWS Lambda is a serverless compute service that executes code in response to events without

requiring the user to manage servers. A Python Lambda function was created, deployed, invoked using a test event, and verified through its successful response.

Amazon RDS

Amazon Relational Database Service (RDS) is a managed service for relational databases. A MySQL RDS database instance was configured and created for the demonstration, verified in the Available state, and later deleted so that the temporary assignment resource was not left running.

Elastic Load Balancing

Elastic Load Balancing distributes incoming application traffic across registered targets. An Application Load Balancer (ALB) was configured with an HTTP listener, a target group, and the EC2 instance running NGINX. The ALB DNS endpoint was tested successfully. The ALB and target group were subsequently deleted.

Amazon ECS

Amazon Elastic Container Service (ECS) is a managed container orchestration service used to run and manage containerized applications. A Fargate-based ECS cluster was created as a basic demonstration of ECS. The cluster was successfully created and verified from the ECS console. No additional application task was required for this basic service demonstration.

Code / Implementation:

The practical implementation was primarily performed through the AWS Management Console. The following steps document the actual workflow followed.

EC2 – Create and Configure Virtual Machine

1. Opened Amazon EC2 in the AWS Management Console and selected the Mumbai region (ap-south-1).
2. Selected an AMI and suitable instance type.
3. Configured a key pair for secure instance access.
4. Configured the security group with required network rules, including SSH access and HTTP access for the NGINX demonstration.
5. Launched the EC2 instance and verified that its state became Running.

EC2 – Connect and Verify

The instance was accessed using EC2 Instance Connect. Basic VM verification commands were executed.

```
id
hostname
uname -a
```

EC2 – Configure NGINX

NGINX was installed/configured as the web server and its service was started and enabled.

```
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx
```

The NGINX service was verified locally and through the EC2 web endpoint.

EC2 – Monitoring

6. EC2 instance monitoring information was inspected from the AWS console.
7. CloudWatch CPU utilization information was checked to demonstrate monitoring.
8. The instance status checks and resource state were reviewed before cleanup.

Amazon S3 Demonstration

9. Opened Amazon S3 and created a general-purpose bucket with public access blocked.
10. Uploaded a small test text file to the bucket.
11. Verified the uploaded object from the bucket's Objects section.
12. Deleted the object and the assignment bucket during cleanup.

AWS Lambda Demonstration

A Python Lambda function was created with the following handler:

```
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "body": "Hello from AWS Lambda!"
    }
```

The function was deployed and invoked with a test event. The execution returned HTTP-style status code 200 and the message shown above. The temporary Lambda function was deleted during cleanup.

Amazon RDS Demonstration

13. Selected Amazon RDS and opened the Create database workflow.
14. Selected MySQL as the database engine and configured the temporary assignment database.
15. Waited for the database to reach the Available state.
16. Deleted the temporary RDS instance after capturing the required evidence.

Elastic Load Balancing Demonstration

17. Created an EC2 target group using HTTP on port 80.
18. Registered the running EC2 instance as the target.
19. Created an internet-facing Application Load Balancer.
20. Configured an HTTP listener on port 80 and forwarded traffic to the target group.
21. Verified the target as healthy and accessed the NGINX response using the ALB DNS name.
22. Deleted the ALB and target group during final cleanup.

Amazon ECS Demonstration

23. Opened Amazon ECS and selected Clusters.
24. Configured a Fargate-based ECS cluster.
25. Created the ECS cluster successfully.
26. Verified the cluster from the ECS console.
27. No additional task workload was launched for the basic service demonstration.

Output

The following screenshots are the selected evidence for the implementation. Insert the original high-resolution screenshots in the indicated positions. The numbering below follows the screenshots already captured during the practical.

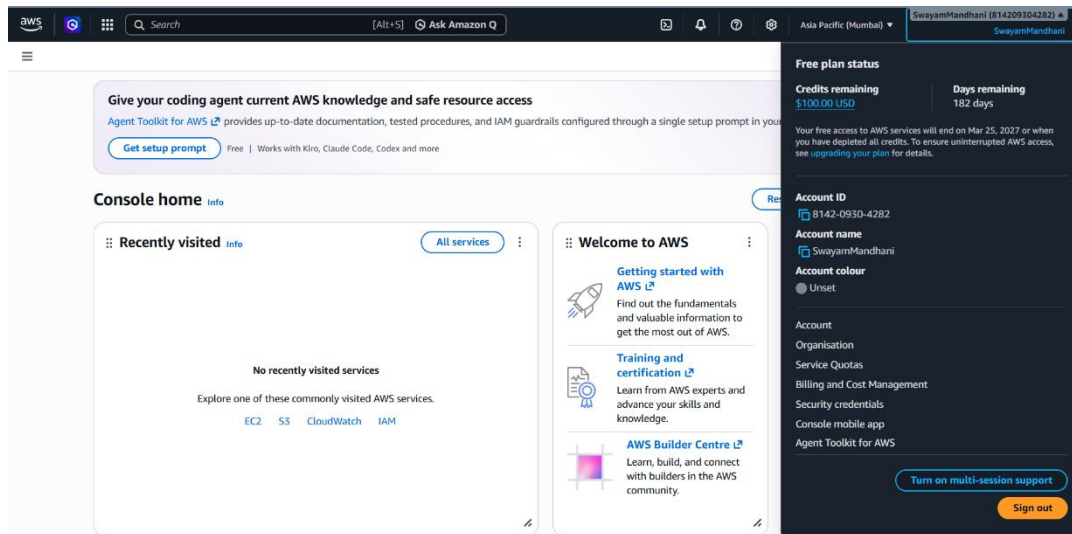


Figure 1: AWS Console region selection

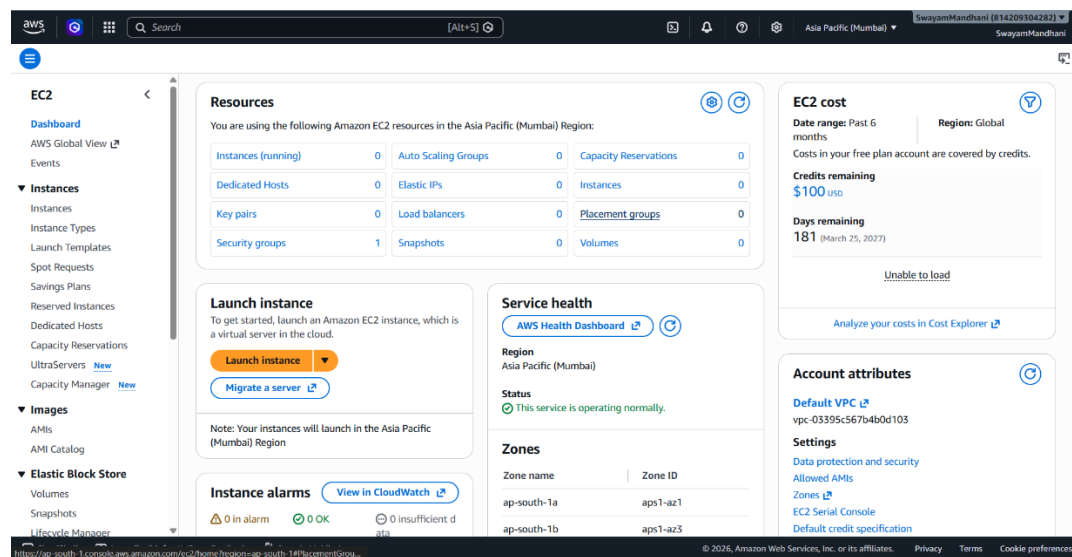


Figure 2: Amazon EC2 dashboard

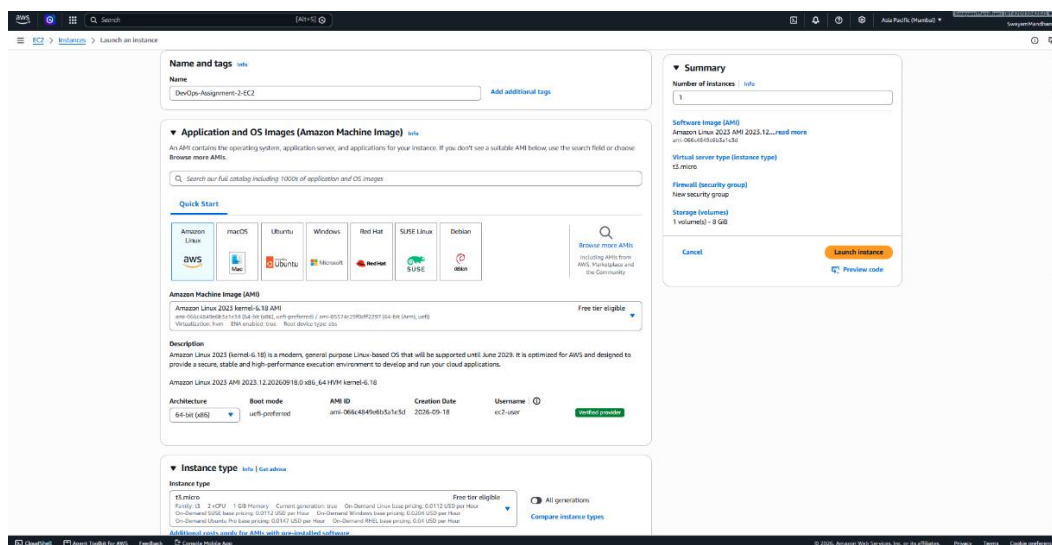


Figure 3: EC2 AMI and instance-type configuration

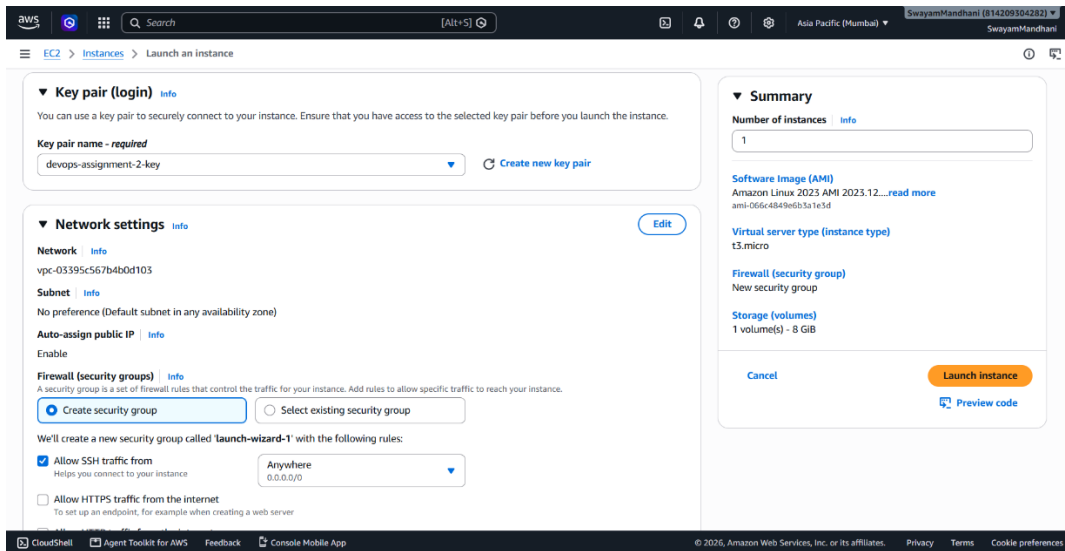


Figure 4: EC2 key-pair configuration

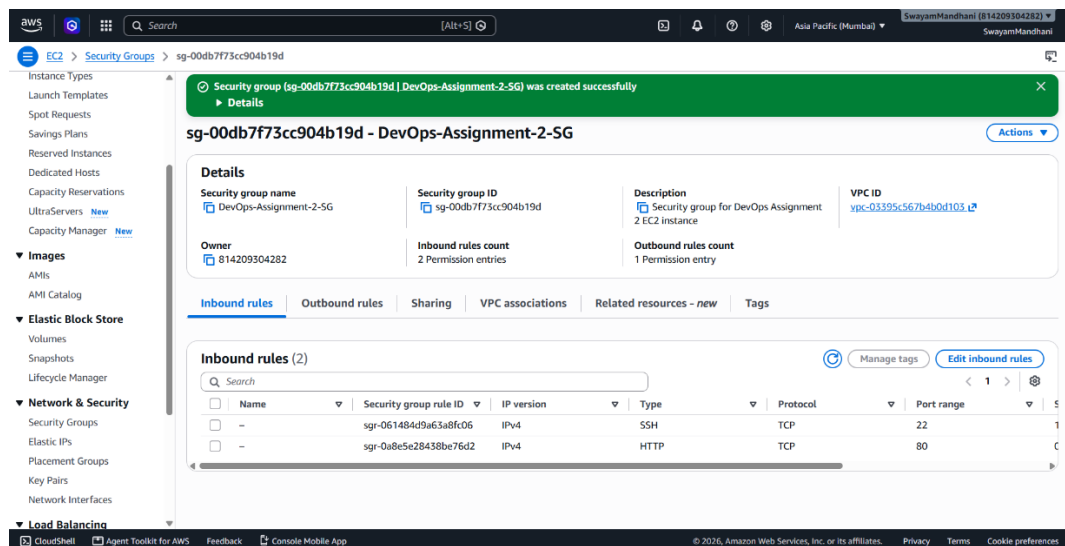


Figure 5: EC2 security-group configuration

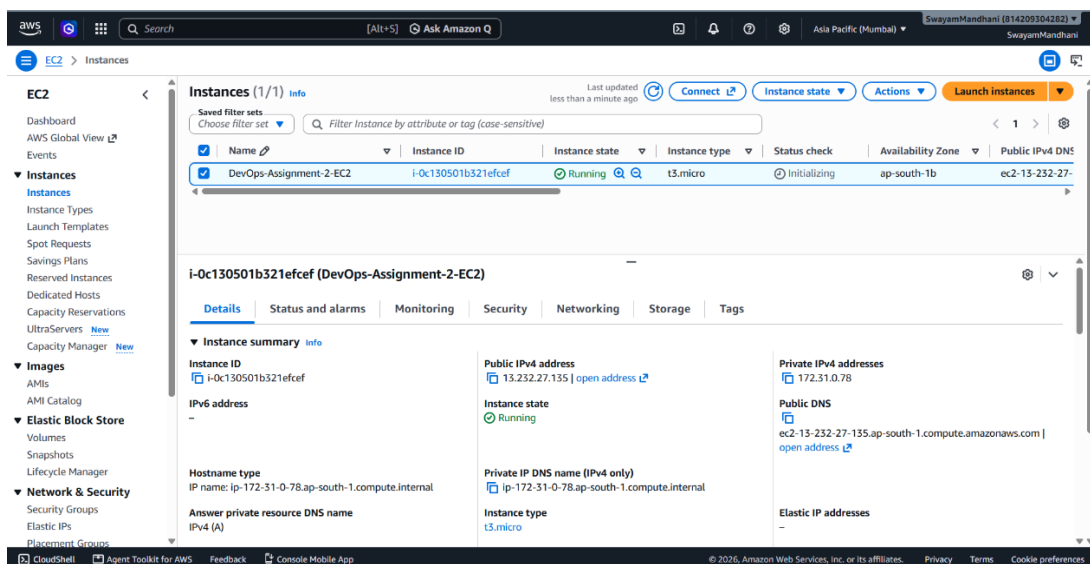


Figure 6: EC2 instance in Running state

```
aws
[ec2-user@ip-172-31-0-78 ~]$ whoami
ec2-user
[ec2-user@ip-172-31-0-78 ~]$ hostname
ip-172-31-0-78.ap-south-1.compute.internal
[ec2-user@ip-172-31-0-78 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-0-78 ~]$
```

I-0c130501b321efcef (DevOps-Assignment-2-EC2)
PublicIPs: 13.232.27.135 PrivateIPs: 172.31.0.78

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

Figure 7: EC2 VM connection and verification

```
aws
Verifying : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch
Installed:
  generic-logos-httpd-10.0.0-12.amzn2023.0.3.noarch      gperftools-lib-2.9.1-1.amzn2023.0.3.x86_64      libunwind-1.4.0-5.amzn2023.0.3.x86_64
  nginx-1:1.30.4-1.amzn2023.0.1.x86_64                  nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64      nginx-filesystem-1:1.30.4-1.amzn2023.0.1.noarch
  nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
[ec2-user@ip-172-31-0-78 ~]$ sudo dnf install -y nginx
Last metadata expiration check: 0:00:24 ago on Fri Sep 25 13:18:05 2026.
Package nginx-1:1.30.4-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-0-78 ~]$ sudo systemctl start nginx
[ec2-user@ip-172-31-0-78 ~]$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service - /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-172-31-0-78 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-25 13:18:36 UTC; 14s ago
     Main PID: 26742 (nginx)
       Tasks: 3 (limit: 1059)
      Memory: 3.4M
         CPU: 0ms
    CGroup: /system.slice/nginx.service
            └─26742 "nginx: master process /usr/sbin/nginx"
              └─26743 "nginx: worker process"
                └─26744 "nginx: worker process"

Sep 25 13:18:36 ip-172-31-0-78.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Sep 25 13:18:36 ip-172-31-0-78.ap-south-1.compute.internal nginx[26740]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Sep 25 13:18:36 ip-172-31-0-78.ap-south-1.compute.internal nginx[26740]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Sep 25 13:18:36 ip-172-31-0-78.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-172-31-0-78 ~]$
```

I-0c130501b321efcef (DevOps-Assignment-2-EC2)
PublicIPs: 13.232.27.135 PrivateIPs: 172.31.0.78

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

Figure 8: NGINX service running on EC2

```
aws
Sep 25 13:18:36 ip-172-31-0-78.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Sep 25 13:18:36 ip-172-31-0-78.ap-south-1.compute.internal nginx[26740]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Sep 25 13:18:36 ip-172-31-0-78.ap-south-1.compute.internal nginx[26740]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Sep 25 13:18:36 ip-172-31-0-78.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-172-31-0-78 ~]$ curl http://localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>
<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
to engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>
<p>Thank you for using nginx.</em></p>
</body>
</html>
[ec2-user@ip-172-31-0-78 ~]$
```

I-0c130501b321efcef (DevOps-Assignment-2-EC2)
PublicIPs: 13.232.27.135 PrivateIPs: 172.31.0.78

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

Figure 9: EC2 web output

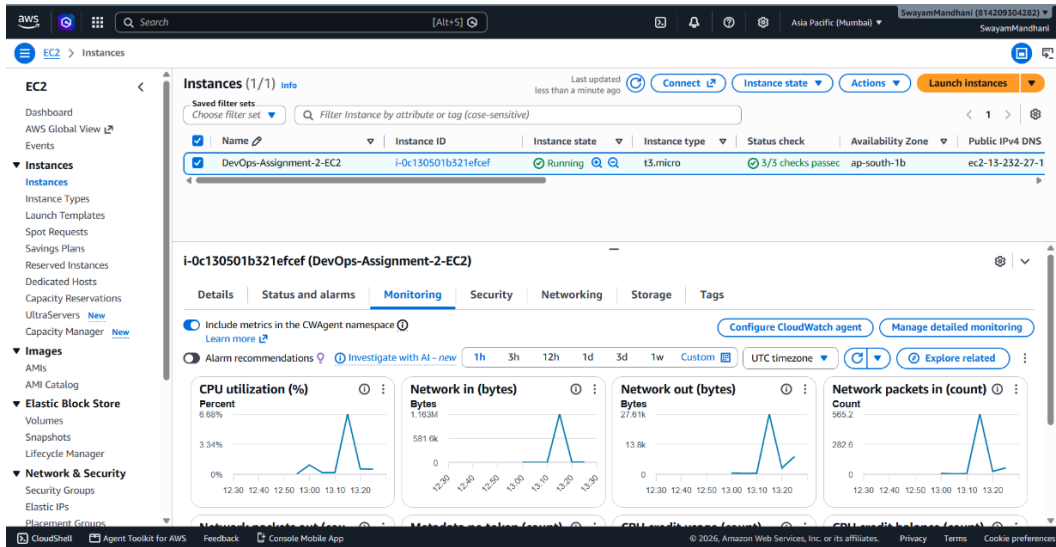


Figure 10: EC2 monitoring

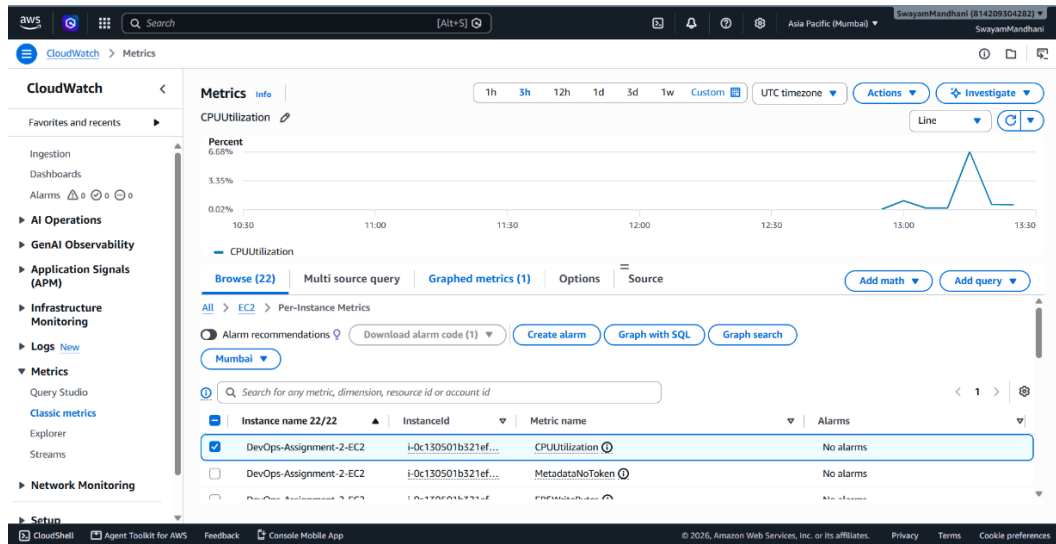


Figure 11: CloudWatch CPU monitoring

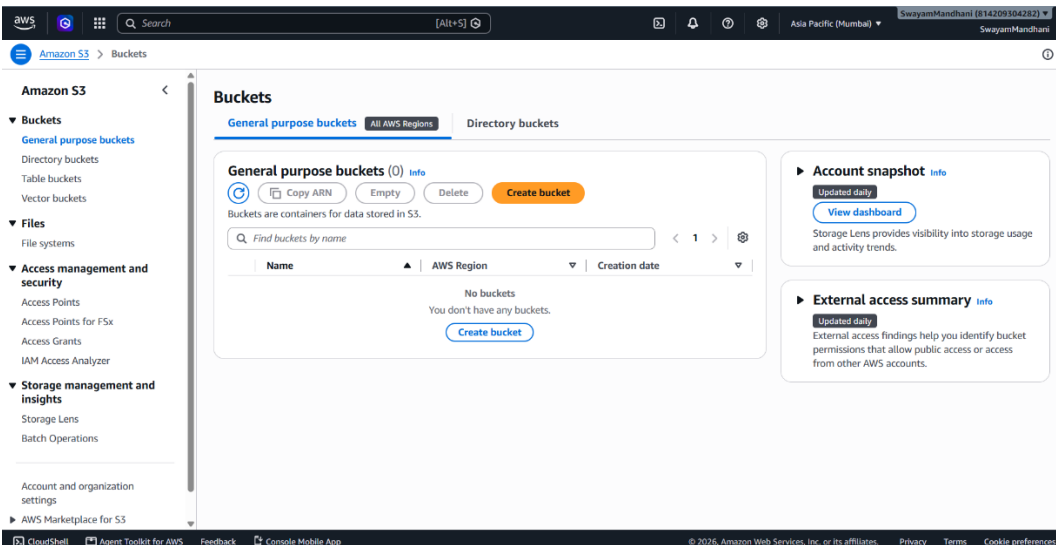


Figure 12: Amazon S3 dashboard

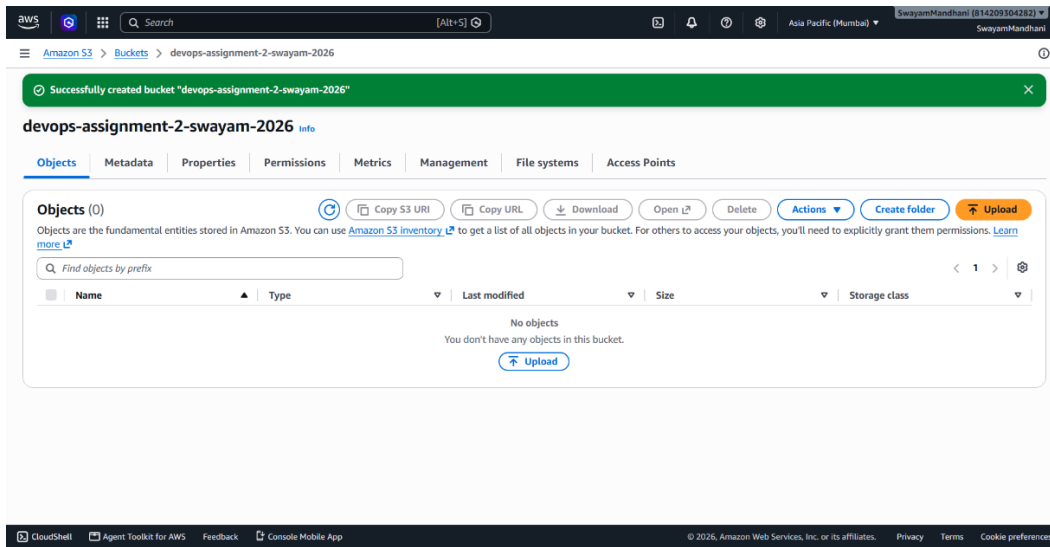


Figure 13: S3 bucket created

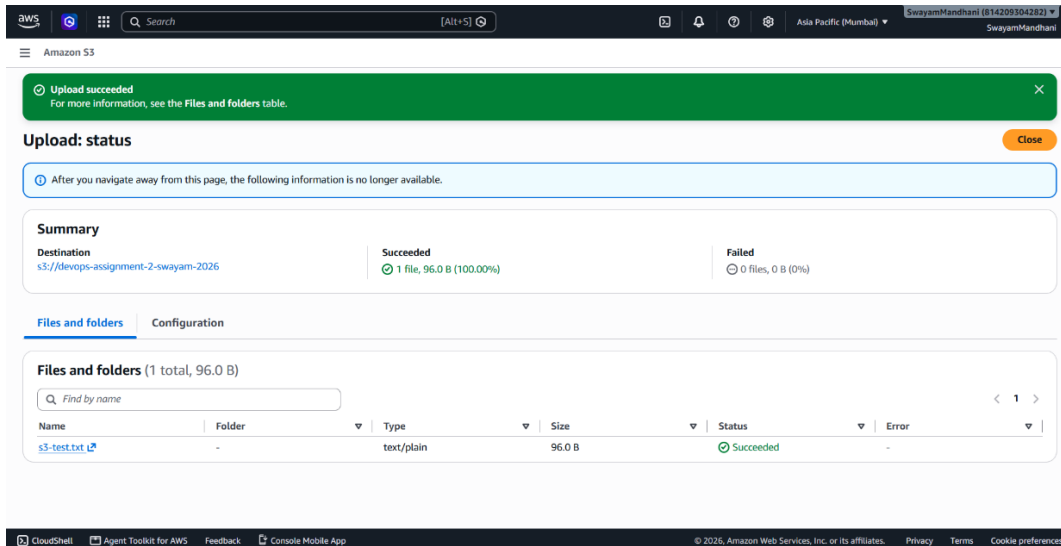


Figure 14: S3 object uploaded

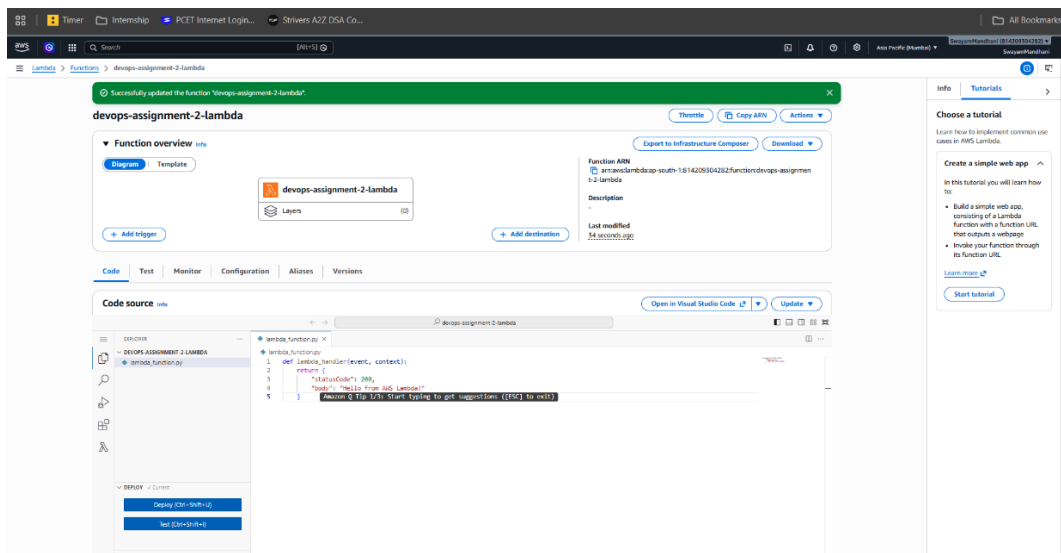


Figure 15: AWS Lambda code

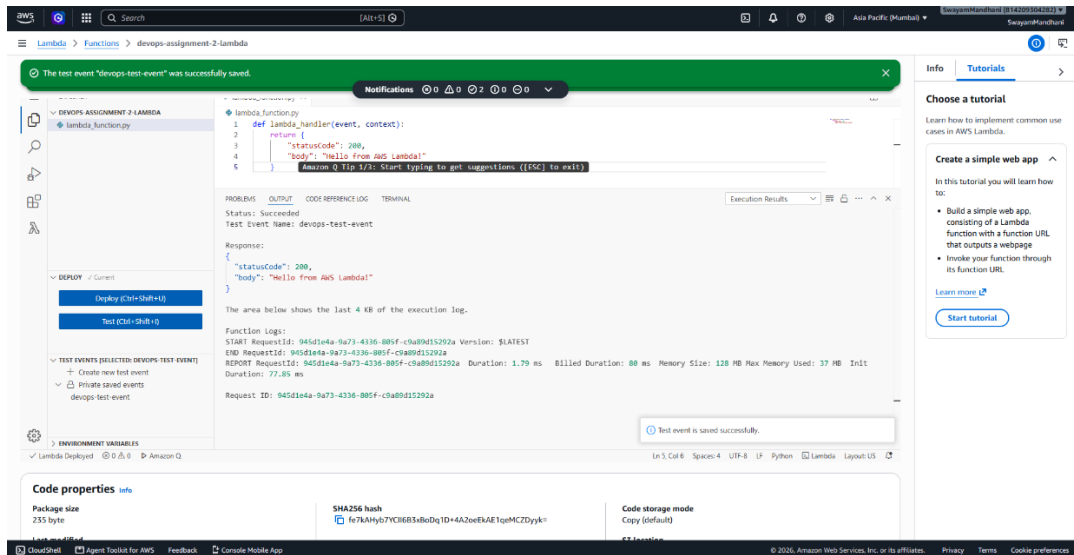


Figure 16: AWS Lambda successful test

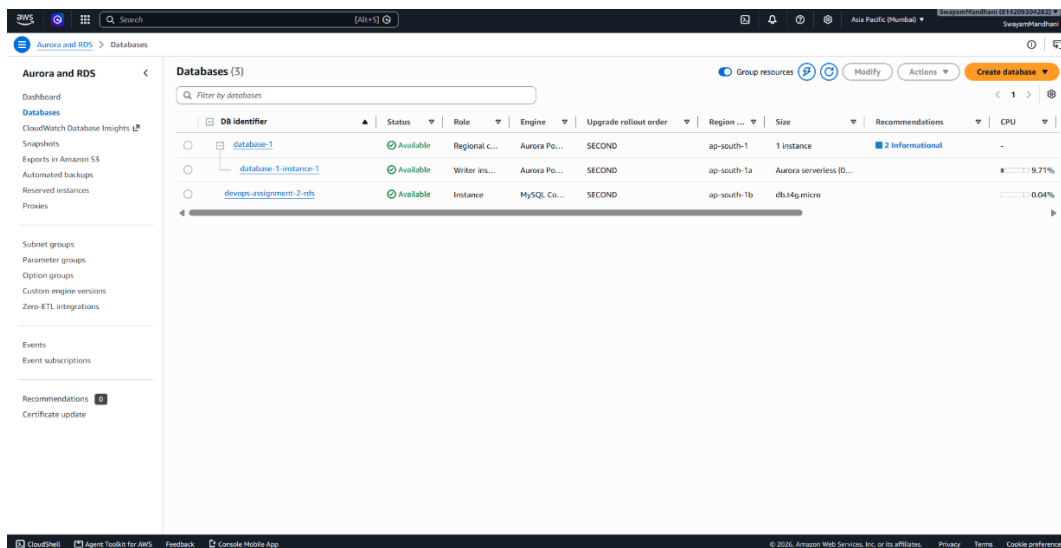


Figure 17: Amazon RDS database in Available state

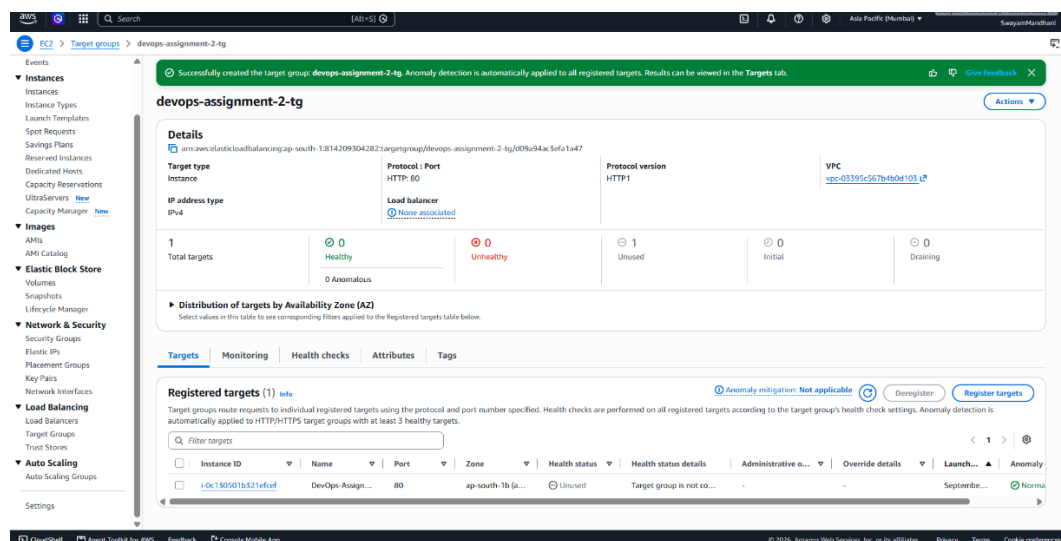


Figure 18: ELB target group

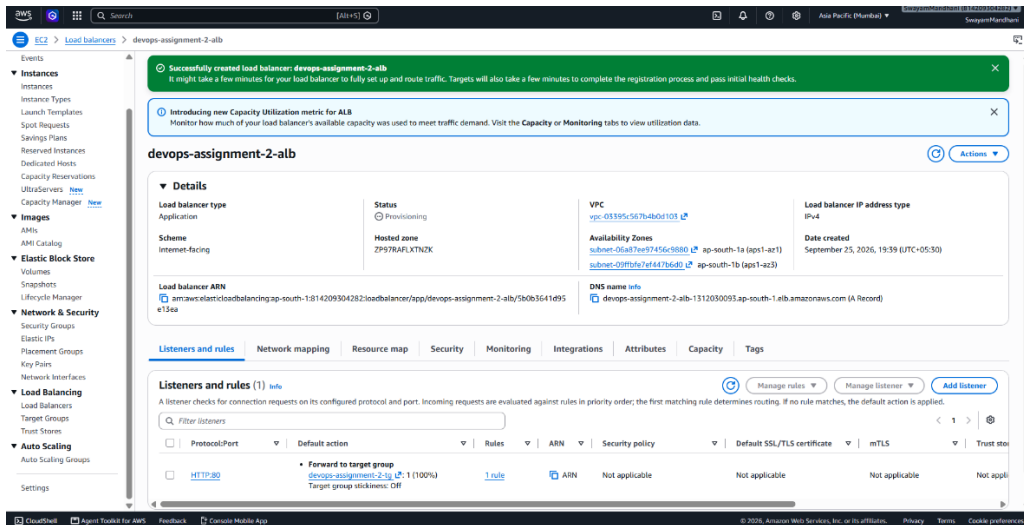


Figure 19: Application Load Balancer configuration

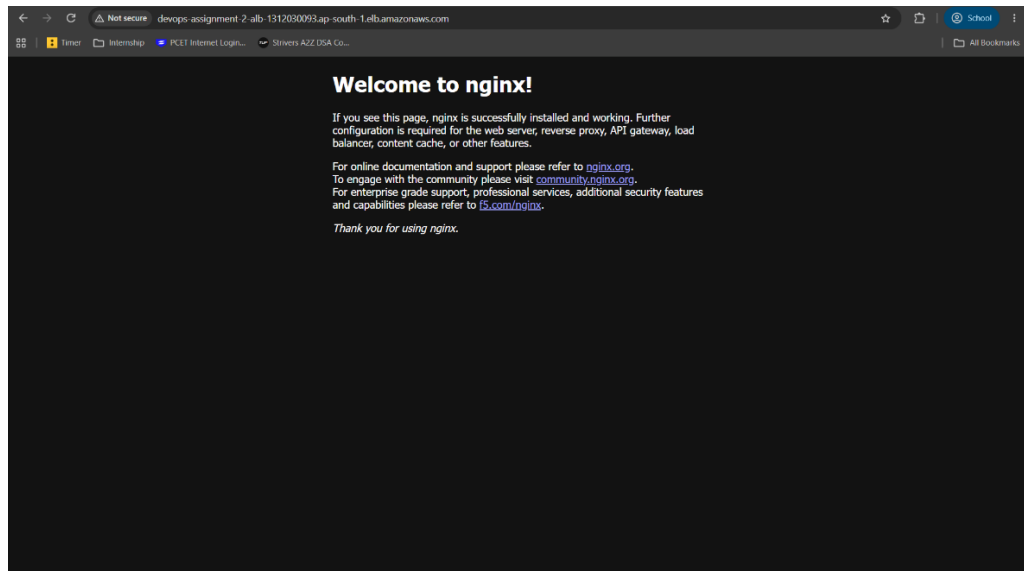


Figure 20: Working Application Load Balancer endpoint

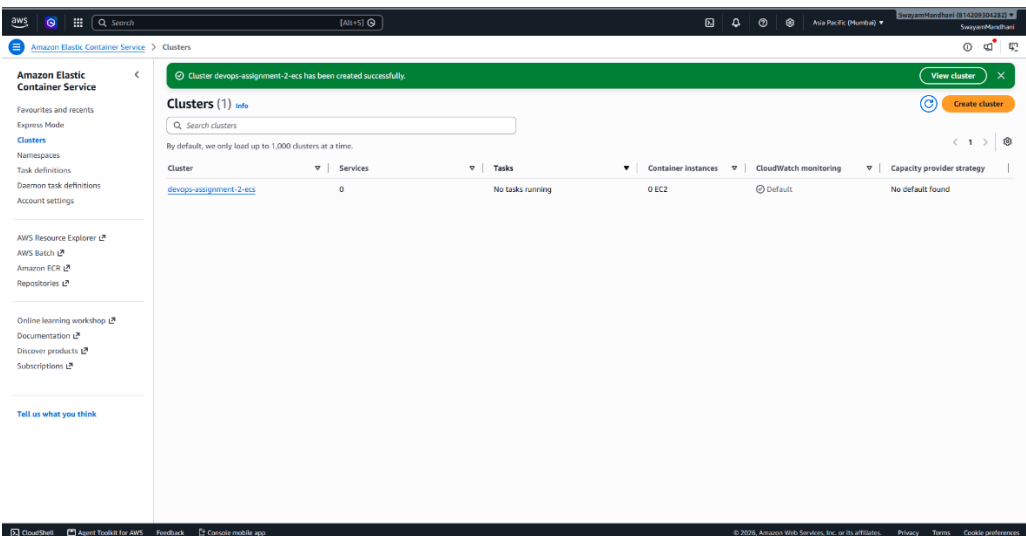


Figure 21: Elastic Container Service (ECS) Cluster

Conclusion:

In this assignment, I studied major AWS cloud computing services including Amazon EC2, Amazon S3, AWS Lambda, Amazon RDS, Elastic Load Balancing, and Amazon ECS. I practically created and configured an EC2 virtual machine, connected to it using EC2 Instance Connect, deployed and verified NGINX, monitored the instance using EC2 and CloudWatch, and terminated the instance after completing the demonstration. I also performed basic demonstrations of S3, Lambda, RDS, and ELB. ECS was studied and a Fargate cluster creation was attempted, but AWS returned a service-linked-role error; no ECS workload was left running. The exercise provided practical understanding of cloud compute, storage, serverless execution, managed databases, load balancing, and container orchestration services.

References:

1. Amazon EC2 User Guide – <https://docs.aws.amazon.com/ec2/>
2. Amazon S3 User Guide – <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>
3. AWS Lambda Developer Guide – <https://docs.aws.amazon.com/lambda/>
4. Amazon RDS User Guide – <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/>
5. Elastic Load Balancing User Guide – <https://docs.aws.amazon.com/elasticloadbalancing/>
6. Amazon ECS Developer Guide – <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/>

Submission Links:

GitHub Repository Link: <https://github.com/SwayamMandhani06/DevOps/tree/main/Assignment-2>

Hosted / Output Link: NA – AWS Console Based